Team Juggernauts

DIY Robot Challenge 2026

ROS 2 Humble · Jetson Orin Nano · Zone Nav Stack

This guide covers: workspace build · launching the competition stack · pre-run health checks · live monitoring · bag recording · CSV logging · bag replay · debug tools · field troubleshooting decision tree.

1 · Workspace Build (Local Dev Machine)

The repo has two workspaces: the main workspace in the repo root (zone nav, bringup, mux) and a third-party workspace for SLAM libraries. phoenix6 (CTRE motor vendor) is robot-only — skip it for local builds.

1.1 One-time Setup

Source ROS 2 Humble

```
source /opt/ros/humble/setup.bash
# Add to ~/.bashrc to avoid repeating every session:
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
```

1.2 Build Zone Nav Stack (recommended for debugging)

scripts/env.sh must be sourced first

```
cd /path/to/DIY-Challenge-Repo
source /opt/ros/humble/setup.bash

colcon build --symlink-install \
  --packages-select \
    diy_zone_nav \
    diy_cmd_vel_mux \
    diy_estop_controller \
    challenge_bringup \
    diy_robot_description

source install/setup.bash
```

■ NOTE: diy_motor_control_legacy requires phoenix6 (CTRE vendor lib — robot only). Always skip it on dev machines.

1.3 Build Third-Party SLAM Stack (if needed for replay)

Build separately in third_party_ws/

```
cd /path/to/DIY-Challenge-Repo/third_party_ws
colcon build --symlink-install
source install/setup.bash
```

1.4 Full On-Robot Build (Jetson)

On Jetson Orin — builds everything including motor driver

```
cd /path/to/DIY-Challenge-Repo
source /opt/ros/humble/setup.bash
colcon build --symlink-install
source install/setup.bash
```

■ WARNING: Always run source install/setup.bash after every build. Missing this step is the most common cause of "package not found" errors.

2 · Environment & Profiles

scripts/env.sh sets all DIY_* environment variables from the selected profile before launching. Profiles live in profiles/ and define which hardware and algorithms are enabled for that machine.

Source environment for a profile

```
source scripts/env.sh jetson       # full hardware stack (Jetson Orin)
source scripts/env.sh laptop       # software-only / replay mode
source scripts/env.sh sim          # Gazebo simulation
```

Key DIY_* Variables

Variable	Description
DIY_ROBOT_PROFILE	Active profile name (jetson/laptop/sim)
DIY_USE_NAV2	true/false — enable Nav2 navigation stack
DIY_USE_ZONE_NAV	true/false — enable zone nav state machine
DIY_USE_LOCALIZATION	true/false — enable FAST-LIO2 + NDT pipeline
DIY_USE_HESAI	true/false — enable Hesai QT64 lidar driver
DIY_USE_MOTOR_DRIVER	true/false — enable CTRE motor driver
DIY_MUX_MODE	Initial mux mode: JOYSTICK/AUTONOMOUS
DIY_FASTLIO_CONFIG	FAST-LIO2 config YAML filename

3 · Launching the Competition Stack

3.1 Full Competition Launch (run_robot.sh)

This is the primary launch script for arena runs. It sources the environment, then calls challenge_master.launch.py with all arguments resolved from the profile.

scripts/run_robot.sh

```
# On Jetson — full hardware stack
./scripts/run_robot.sh jetson

# On laptop — software-only (requires bag file set in profile)
./scripts/run_robot.sh laptop
```

3.2 Direct Launch via challenge_master.launch.py

Use this when you need to override individual arguments without changing the profile file.

ros2 launch — manual argument override

```
source /opt/ros/humble/setup.bash
source install/setup.bash

ros2 launch challenge_bringup challenge_master.launch.py \
    use_joystick:=false \
    use_nav2:=true \
    use_zone_nav:=true \
    use_localization:=true \
    use_hesai:=true \
    use_motor_driver:=true \
    use_realsense:=false \
    use_micro_ros:=true \
    mux_mode:=AUTONOMOUS \
    fastlio_config:=fast_lio_hesai_qt64.yaml
```

3.3 Zone Nav Only (without full bringup)

Launch zone nav subsystem in isolation

```
ros2 launch diy_zone_nav zone_nav.launch.py \
    use_localization:=true \
    launch_gate:=true
```

■ NOTE: When launched from challenge_master, zone_nav.launch.py receives launch_gate:=false to avoid duplicating the lidar_odom_gate_node which the master already starts.

3.4 Launch Arguments Reference

Argument	Type	Description
use_joystick	true/false	Enable joystick driver node
use_nav2	true/false	Enable Nav2 navigation stack
use_zone_nav	true/false	Enable zone nav state machine
use_localization	true/false	Enable FAST-LIO2 + NDT + lidar gate
use_hesai	true/false	Enable Hesai QT64 lidar driver
use_motor_driver	true/false	Enable CTRE TalonFX motor driver

use_realsense	true/false	Enable RealSense depth camera
use_micro_ros	true/false	Enable micro-ROS (IMU, wheel enc.)
mux_mode	string	Initial mux mode (JOYSTICK / AUTONOMOUS)
fastlio_config	string	FAST-LIO2 config YAML filename
launch_gate	true/false	Launch lidar_odom_gate_node here (default true)
use_rviz	true/false	Launch RViz2 for visualization

4 · Pre-Run Checklist (Before Every Arena Run)

4.1 Automated Health Check

Run health_check_zone_nav.sh after launching the stack. All 11 checks must PASS before the robot moves.

scripts/health_check_zone_nav.sh

```
bash scripts/health_check_zone_nav.sh

# Expected output:
#   [PASS] /nav_mode is publishing
#   [PASS] /mux_mode is publishing
#   [PASS] /odometry/filtered is publishing
#   [PASS] /ndt_fitness_score is publishing
#   [PASS] /cmd_vel_safe is publishing
#   [PASS] Service /global_costmap/global_costmap/set_parameters is available
#   [PASS] Service /local_costmap/local_costmap/set_parameters is available
#   [PASS] Service /cmd_vel_mux_node/set_parameters is available
#   [PASS] /nav_mode value readable
#   [PASS] /mux_mode value readable
#   [PASS] /estop_active is False (safe to run)
#   Passed: 11  Failed: 0
```

■ CRITICAL: Do NOT start a run if any check FAILS. A silent topic means a crashed node — find and fix it first.

4.2 Manual Pre-Run Checklist

■	E-stop button is released (LED off)
■	Joystick is connected and responsive
■	Mux mode is set to JOYSTICK for manual verification, then switch to AUTONOMOUS
■	Robot is placed at start line, pointing toward first waypoint
■	zone_waypoints.yaml coordinates match the actual arena layout
■	ndt_fitness_score < 0.5 (good localization before start)
■	Recording tools are started (record_zone_nav.sh + zone_nav_logger.py)
■	Battery is fully charged
■	All team members clear of the robot path

5 · Live Monitoring Tools

5.1 Zone Nav Dashboard (inspect_zone_nav.sh)

Displays a live terminal dashboard refreshing every 1 second. Shows nav_mode, mux_mode, estop, NDT fitness, odom position, and cmd_vel.

scripts/inspect_zone_nav.sh

```
bash scripts/inspect_zone_nav.sh

# Run in a separate terminal from the robot launch.
# Press Ctrl+C to exit.
```

5.2 rqt Tools

Tool	Command	Use
rqt_graph	rqt_graph	Node/topic connection graph — spot disconnected nodes
rqt_console	rqt_console	Filtered log viewer — filter by node name or severity
rqt_plot	rqt_plot	Real-time topic plotting (basic — use PlotJuggler for serious analysis)
rqt_tf_tree	rqt > Plugins > TF Tree	Verify TF frames are publishing

5.3 RViz2 Key Displays

Display Type	Topic	Purpose
Map	/map	Occupancy grid (pre-built map for NDT)
Path	/plan	Nav2 planned path to goal
Pose	/odometry/filtered	Robot position estimate (EKF output)
Costmap (global)	/global_costmap/costmap	Global cost layer
Costmap (local)	/local_costmap/costmap	Local obstacle layer (VoxelLayer)
PointCloud2	/hesai/points	Raw lidar scan (heavy — disable if slow)
TF	—	Visualize all active TF frames

Launch RViz2 standalone

```
rviz2 -d src/diy_robot_description/rviz/default.rviz
```

5.4 Quick Topic Checks (One-liners)

Useful ros2 topic commands during a run

```
# Check nav state machine mode
ros2 topic echo --once /nav_mode

# Check velocity arbitration mode
ros2 topic echo --once /mux_mode

# Check NDT localization quality (lower = better; >0.8 = degraded)
ros2 topic echo --once /ndt_fitness_score

# Check what velocity is being commanded to motors
```

```
ros2 topic echo --once /cmd_vel_safe

# Check active speed limit
ros2 topic echo --once /speed_limit

# Monitor topic publish rates (4-second window)
ros2 topic hz /nav_mode --window 4
ros2 topic hz /odometry/filtered --window 4
ros2 topic hz /ndt_fitness_score --window 4
```

5.5 PlotJuggler (install once)

Install and launch

```
sudo apt install ros-humble-plotjuggler-ros
plotjuggler
```

In PlotJuggler: File → Start ROS2 Topic Subscriber, then drag topics onto the plot canvas. Drag /ndt_fitness_score, /cmd_vel_safe/linear/x, and /nav_mode onto the same time axis to correlate mode changes with speed commands during a run.

6 · Recording Tools

Always record during arena runs. If something goes wrong, the recording lets you replay and debug the exact sequence of events without needing the robot to be present.

6.1 Bag Recorder — record_zone_nav.sh

Records 20 zone nav state topics (no lidar point clouds, no camera). A full run fits in ~5 MB vs. GBs for a full bag. Output: ~/bags/zone_nav_<label>_<timestamp>/

scripts/record_zone_nav.sh

```
# Basic usage — label defaults to "run"
bash scripts/record_zone_nav.sh

# With a descriptive label
bash scripts/record_zone_nav.sh --label arena_run_01

# Run in background while monitoring in another terminal
bash scripts/record_zone_nav.sh --label arena_run_01 &

# Stop recording
Ctrl+C  (or kill the background PID)

# Output location
ls ~/bags/zone_nav_arena_run_01_*/
```

Recorded Topics

Topic	Content
/nav_mode	Zone nav state machine mode (INIT/NORMAL_NAV/SLOW_NAV/BLIND_DRIVE/...)
/green_light	Race start trigger
/mux_mode	Velocity arbitration mode (JOYSTICK/AUTONOMOUS/BLIND_DRIVE/ESTOP_LOCK)
/cmd_vel_zone_nav	Zone nav velocity command
/cmd_vel_nav	Nav2 velocity command
/cmd_vel_joy	Joystick velocity command
/cmd_vel_safe	Final velocity to motors (mux output)
/speed_limit	Active speed cap (nav2_msgs/SpeedLimit)
/estop_active	Hardware e-stop state
/ndt_fitness_score	NDT match quality (Float32, lower=better)
/odometry/filtered	EKF fused position estimate
/lidar_odometry	FAST-LIO2 raw odometry
/lidar_odometry_gated	Lidar odom gated off during BLIND_DRIVE
/tf + /tf_static	Transform frames for rviz2 replay
/diagnostics	Node diagnostic messages

/rosout	All ROS log output

6.2 CSV Flight Data Logger — zone_nav_logger.py

A standalone Python node that writes a timestamped 13-column CSV file for post-run analysis. Writes immediately on any state transition (nav_mode/mux_mode/estop changes); throttles high-frequency topics to 5 Hz. Output: ~/bags/zone_nav_log_<timestamp>.csv

scripts/zone_nav_logger.py

```
# Run in a separate terminal alongside the stack
python3 scripts/zone_nav_logger.py

# Output location
ls ~/bags/zone_nav_log_*.csv
```

CSV Column Reference

Column	Description
wall_time_s	Unix wall-clock timestamp (seconds)
ros_time_s	ROS clock timestamp (seconds)
nav_mode	Current zone nav state machine mode string
mux_mode	Current velocity mux mode string
estop_active	True/False — hardware e-stop state
ndt_fitness	NDT match fitness score (Float32, lower=better)
odom_x	EKF position X in map frame (meters)
odom_y	EKF position Y in map frame (meters)
odom_yaw_deg	EKF heading angle (degrees)
speed_limit_ms	Active speed cap (m/s, 0.0 = no limit)
cmd_vel_linear_x	Final commanded forward velocity (m/s)
cmd_vel_angular_z	Final commanded angular velocity (rad/s)
event	Optional label — state transition trigger description

Analyse in Python / pandas

Post-run analysis snippet

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("~/bags/zone_nav_log_20260801_143022.csv")

# Plot speed vs time, coloured by nav_mode
fig, ax = plt.subplots(figsize=(14, 4))
for mode, grp in df.groupby("nav_mode"):
    ax.plot(grp["wall_time_s"], grp["cmd_vel_linear_x"], label=mode)
ax.set_xlabel("Time (s)")
ax.set_ylabel("Linear velocity (m/s)")
ax.legend()
plt.tight_layout()
plt.show()
```

```
# Find all BLIND_DRIVE events
blind = df[df["nav_mode"] == "BLIND_DRIVE"]
print(f"BLIND_DRIVE entries: {len(blind)}")
print(blind[["wall_time_s", "ndt_fitness", "odom_x", "odom_y"]].head(20))
```

Open in PlotJuggler

Load CSV in PlotJuggler

```
plotjuggler
# File → Load Data File → select the .csv
# Drag columns onto plot canvas
# Suggested layout:
#   Row 1: cmd_vel_linear_x, speed_limit_ms
#   Row 2: ndt_fitness
#   Row 3: odom_x, odom_y  (as XY scatter for path trace)
```

7 · Bag Replay (Offline Debugging)

Bag replay lets you re-run a recorded session through the full software stack without the robot. Use this to reproduce bugs, tune parameters, or verify fixes before the next arena session.

7.1 replay_bag.sh

scripts/replay_bag.sh

```
# Full replay at real speed
bash scripts/replay_bag.sh ~/bags/zone_nav_arena_run_01_20260801_143022 laptop

# Slow-motion replay (0.5x speed — easier to follow state transitions)
bash scripts/replay_bag.sh ~/bags/zone_nav_arena_run_01_* laptop --rate 0.5

# Replay without Nav2 (just localization + zone nav — faster startup)
bash scripts/replay_bag.sh ~/bags/zone_nav_arena_run_01_* laptop --no-nav2
```

7.2 Manual Bag Replay

Play a bag directly (stack must be running separately)

```
# In terminal 1 — start the stack (no hardware drivers)
ros2 launch challenge_bringup challenge_master.launch.py \
    use_hesai:=false use_motor_driver:=false \
    use_micro_ros:=false use_rviz:=true

# In terminal 2 — play the bag
ros2 bag play ~/bags/zone_nav_arena_run_01_*/ --clock

# Slow it down
ros2 bag play ~/bags/zone_nav_arena_run_01_*/ --clock --rate 0.3

# Loop it
ros2 bag play ~/bags/zone_nav_arena_run_01_*/ --clock --loop
```

7.3 Inspect Bag Contents

Check what is in a bag before replaying

```
ros2 bag info ~/bags/zone_nav_arena_run_01_*/

# List topics and message counts
# Look for: duration, message count per topic, missing topics
```

■ WARNING: Always use --clock when replaying bags. Without it, nodes use wall-clock time and time-based logic (watchdogs, timeouts) will behave incorrectly.

8 · Injection & Control Tools

These tools let you manually trigger events and change modes at runtime without physical hardware — essential for bench testing and verifying state transitions before arena runs.

8.1 Set Mux Mode — set_mux_mode.sh

scripts/set_mux_mode.sh

```
# Switch to autonomous (Nav2 controls robot)
bash scripts/set_mux_mode.sh AUTONOMOUS

# Switch to joystick control
bash scripts/set_mux_mode.sh JOYSTICK

# Switch to blind drive mode (lidar gated, fixed velocity)
bash scripts/set_mux_mode.sh BLIND_DRIVE

# Show current mode (no argument)
bash scripts/set_mux_mode.sh

# NOTE: ESTOP_LOCK cannot be set manually — hardware e-stop button only
```

8.2 Simulate Green Light — trigger_green_light.sh

Publishes a single Bool:true message on /green_light, causing the zone nav state machine to transition from INIT → NORMAL_NAV. Use this for bench testing without the physical start gate signal.

scripts/trigger_green_light.sh

```
bash scripts/trigger_green_light.sh

# Expected result:
# /nav_mode transitions from INIT → NORMAL_NAV
# /mux_mode transitions from JOYSTICK → AUTONOMOUS
# Robot begins following the Nav2 plan
```

8.3 Manual Topic Publishing

Useful one-liners for testing

```
# Manually publish green light
ros2 topic pub --once /green_light std_msgs/msg/Bool "data: true"

# Manually set a speed limit (m/s)
ros2 topic pub --once /speed_limit nav2_msgs/msg/SpeedLimit \
    "{header: {frame_id: map}, speed_limit: 0.5, percentage: false}"

# Manually trigger e-stop (simulated)
ros2 topic pub --once /estop_active std_msgs/msg/Bool "data: true"

# Release simulated e-stop
ros2 topic pub --once /estop_active std_msgs/msg/Bool "data: false"
```

9 · Step-by-Step Validation Scripts

Use these scripts to validate each layer of the stack in isolation before attempting a full arena run. Run them in order — each builds on the previous layer.

Step 1: test_step1_wheel_odom.sh

Validate wheel odometry from micro-ROS encoders. — Checks /wheel_odom topic is publishing at ~20 Hz.

```
bash scripts/test_step1_wheel_odom.sh
```

Step 2: test_step2_fused_odom.sh

Validate EKF fused odometry. — Checks /odometry/filtered is running and covariance is reasonable.

```
bash scripts/test_step2_fused_odom.sh
```

Step 3: test_step3_fastlio.sh

Validate FAST-LIO2 lidar-inertial odometry. — Checks /lidar_odometry and confirms point cloud is arriving.

```
bash scripts/test_step3_fastlio.sh
```

Step 4: test_step4_motion_plan.sh

Validate Nav2 motion planning. — Sends a test goal and checks that a plan is computed and the robot starts moving.

```
bash scripts/test_step4_motion_plan.sh
```

■ TIP: Run each test script in a separate terminal from the running stack. The stack must already be launched before running any test script.

10 · Field Troubleshooting Decision Tree

Use this decision tree when something goes wrong during an arena run. Start at the top and work down.

10.1 Robot Not Moving at All

1. **Check /estop_active** — if True, release the hardware e-stop button
2. **Check /mux_mode** — if ESTOP_LOCK, release e-stop; if JOYSTICK, switch to AUTONOMOUS
3. **Check /nav_mode** — if INIT, green light was not received; run trigger_green_light.sh
4. **Check /cmd_vel_safe** — if all zeros but /cmd_vel_nav has values, mux is blocking
5. **Check motor driver node** — run: ros2 node list | grep motor
6. **Run health_check_zone_nav.sh** — any FAIL indicates a crashed node

10.2 Robot Moving Wrong Direction or Wrong Speed

1. **Check /speed_limit** — a very low speed_limit will cap velocity unexpectedly
2. **Check /nav_mode** — SLOW_NAV mode halves speed; check if robot entered wrong zone
3. **Check zone waypoints** — zone_waypoints.yaml coordinates may be wrong for this arena
4. **Check /odometry/filtered** — drifted position makes Nav2 plan wrong paths
5. **Check /ndt_fitness_score** — if > 0.8, localization is degraded; BLIND_DRIVE may activate

10.3 NDT Localization Degraded (fitness > 0.5)

1. **Check lidar is publishing** — ros2 topic hz /hesai/points --window 4
2. **Check /lidar_odometry** — FAST-LIO2 may have crashed; check node list
3. **Check /lidar_odometry_gated** — if stuck in BLIND_DRIVE, gate is closed; check /nav_mode
4. **Check map file** — map must match the actual arena; wrong map = bad fitness everywhere
5. **Check lidar mounting** — any physical shift in the lidar breaks the extrinsic calibration

10.4 Zone Nav Stuck in Wrong Mode

1. **Check /nav_mode topic** — confirm the current mode string
2. **BLIND_DRIVE stuck**: fitness must drop below blind_drive_exit_fitness_threshold (0.8) to exit; check NDT
3. **SLOW_NAV stuck**: robot must leave the slow zone radius; check zone_waypoints.yaml coordinates
4. **INIT stuck**: check /green_light topic — may never have received true; run trigger_green_light.sh
5. **Check zone_nav_manager_node logs** — ros2 topic echo /rosout | grep zone_nav

10.5 Bag Replay Not Working

1. Ensure the stack is running with use_hesai:=false use_motor_driver:=false
2. Always include --clock in ros2 bag play command
3. Check bag info — ros2 bag info <bag_path> — confirm expected topics exist
4. Check TF frame timestamps — if TF is stale the robot will not appear in RViz2

5. Reduce --rate to 0.3 if system cannot process messages fast enough

10.6 Quick Diagnosis Commands

Copy-paste these at the arena when something looks wrong

```
# Which nodes are running?
ros2 node list

# Which topics are publishing?
ros2 topic list

# What is zone nav currently doing?
ros2 topic echo --once /nav_mode && \
ros2 topic echo --once /mux_mode && \
ros2 topic echo --once /estop_active && \
ros2 topic echo --once /ndt_fitness_score

# Are all expected topics publishing?
bash scripts/health_check_zone_nav.sh

# Check for error/warning logs
ros2 topic echo /rosout | grep -E "WARN|ERROR|FATAL"

# Zone nav node logs (last 50 lines)
ros2 topic echo /rosout | grep zone_nav_manager
```

11 · Recommended Terminal Layout for Arena Runs

Use a terminal multiplexer (tmux or byobu) to run all tools in one view. Suggested 4-pane layout:

tmux 4-pane setup

```
# Create new session
tmux new-session -s arena

# Split into 4 panes:
# Ctrl+B, "  (horizontal split)
# Ctrl+B, %  (vertical split)

# Pane 1 (top-left): Main stack launch
./scripts/run_robot.sh jetson

# Pane 2 (top-right): Live dashboard
bash scripts/inspect_zone_nav.sh

# Pane 3 (bottom-left): Bag + CSV recorder
bash scripts/record_zone_nav.sh --label arena_run_01
# (new sub-pane below)
python3 scripts/zone_nav_logger.py

# Pane 4 (bottom-right): Health check + ad-hoc commands
bash scripts/health_check_zone_nav.sh
# then use for ros2 topic echo / set_mux_mode / trigger_green_light
```

■ TIP: Start recorders BEFORE the stack reaches INIT mode. Starting them late means you miss the green light and first zone transition events.

Recommended Tool Order

#	Step	Command	When
1	Build	colcon build --packages-select ...	One-time or after code change
2	Launch	./scripts/run_robot.sh jetson	Start full competition stack
3	Record bag	bash scripts/record_zone_nav.sh --label run01	Immediate after launch
4	CSV log	python3 scripts/zone_nav_logger.py	Immediate after launch
5	Health check	bash scripts/health_check_zone_nav.sh	All PASS before moving
6	Dashboard	bash scripts/inspect_zone_nav.sh	Keep open during entire run
7	Green light	bash scripts/trigger_green_light.sh	Start the run
8	Post-run	plotjuggler / pandas CSV	Analyse recording after run

12 · Key Topics & Parameters Quick Reference

12.1 Key Topics

Topic	Type	Description
/nav_mode	std_msgs/String	Zone nav state (INIT/NORMAL_NAV/SLOW_NAV/BLIND_DRIVE/DONE)
/mux_mode	std_msgs/String	Mux mode (JOYSTICK/AUTONOMOUS/BLIND_DRIVE/ESTOP_LOCK)
/green_light	std_msgs/Bool	Race start trigger
/estop_active	std_msgs/Bool	Hardware e-stop state
/ndt_fitness_score	std_msgs/Float32	NDT match quality (lower=better; >0.8=degraded)
/speed_limit	nav2_msgs/SpeedLimit	Active speed cap (percentage=false, 0.0=no limit)
/cmd_vel_zone_nav	geometry_msgs/Twist	Zone nav velocity output
/cmd_vel_nav	geometry_msgs/Twist	Nav2 MPPI velocity output
/cmd_vel_joy	geometry_msgs/Twist	Joystick velocity
/cmd_vel_safe	geometry_msgs/Twist	Final velocity to motors (mux output)
/odometry/filtered	nav_msgs/Odometry	EKF2 fused position estimate
/lidar_odometry	nav_msgs/Odometry	FAST-LIO2 raw odometry
/lidar_odometry_gated	nav_msgs/Odometry	Lidar odom (off during BLIND_DRIVE)

12.2 Key Parameters (Runtime Tunable)

Node	Parameter	Default	Notes
/zone_nav_manager_node	ndt_fitness_gate	0.5	Warn if fitness exceeds this
/zone_nav_manager_node	blind_drive_exit_fitness_threshold	0.8	Exit BLIND_DRIVE when fitness drops below this
/zone_nav_manager_node	zone_entry_radius_m	3.0	Distance to trigger zone entry (m)
/zone_nav_manager_node	slow_nav_speed_limit_ms	0.5	Speed cap in SLOW_NAV zones (m/s)
/cmd_vel_mux_node	mode	AUTO	Override mux mode at runtime
/global_costmap/global_costmap	inflation_layer.inflation_radius	0.35	Global costmap inflation (m)
/local_costmap/local_costmap	voxel_layer.enabled	true	Enable/disable obstacle avoidance

Set a parameter at runtime

```
ros2 param set /zone_nav_manager_node ndt_fitness_gate 0.6
ros2 param set /zone_nav_manager_node slow_nav_speed_limit_ms 0.3
ros2 param set /cmd_vel_mux_node mode AUTONOMOUS
```